



COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	Sep 1, 2026	DATE SUBMITTED:	Sep 1, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

Seatwork 7.2**Algorithm Analysis: Best-Case and Worst-Case Time Complexity****1. PROCEDURE OUTPUT**

Instructions: Analyze each algorithm carefully. For every item, determine the following:

Best-Case Time Complexity

Worst-Case Time Complexity

Brief explanation of how you arrived at your answers.

Use asymptotic notation such as $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, etc.

1. Finding an Element

<pre>for (int i = 0; i < n; i++) { if (arr[i] == target) { return i; } }</pre>	Best: $O(1)$ - The code stops instantly on the first step.. Worst: $O(n)$ - The for loop has to check from 0 to n.
---	--

2. Printing All Elements

<pre>for (int i = 0; i < n; i++) { cout << arr[i] << endl; }</pre>	Best: $O(n)$ - The for loop prints everything. Worst: $O(n)$ - The for loop has to check from 0 to n.
---	---

3. Nested Loops

<pre>for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { cout << i << " " << j << endl; } }</pre>	Best: $O(n^2)$ - A loop inside a loop. Worst: $O(n^2)$ - It runs n times every single time.
---	---

4. Conditional Processing

<pre> if (n > 100) { for (int i = 0; i < n; i++) { cout << arr[i]; } } else { cout << "Small input"; } </pre>	Best: $O(1)$ - It does 1 print. Worst: $O(n)$ - It enters the if block and runs a loop all the way up to n.
---	---

5. Fixed Inner Loop

<pre> for (int i = 0; i < n; i++) { for (int j = 0; j < 10; j++) { cout << i << j; } } </pre>	Best: $O(n)$ - The inside loop stops at 10. Worst: $O(n)$ - Because 10 is a fixed number
---	--

6. Two Consecutive Loops

<pre> for (int i = 0; i < n; i++) { cout << arr[i]; } for (int j = 0; j < n; j++) { cout << arr[j] * 2; } </pre>	Best: $O(n)$ - The loops are separated. Worst: $O(n)$ - It just doubles the work.
---	---

7. Dividing by Two

<pre> int x = n; while (x > 1) { x = x / 2; } </pre>	Best: $O(\log n)$ - Dividing down to 1 shrinks the number. Worst: $O(\log n)$ - Because of $x = x / 2$.
--	--

8. Nested Loop With Different Limits

<pre>for (int i = 0; i < n; i++) { for (int j = 0; j < i; j++) { cout << i << j; } }</pre>	Best: $O(n^2)$ - The inner loop stops at i. Worst: $O(n^2)$ - It means it starts small and gets longer with each step.
--	--

9. Early Termination

<pre>for (int i = 0; i < n; i++) { if (arr[i] == target) { cout << "Found"; break; } }</pre>	Best: $O(1)$ - It exits the loop instantly. Worst: $O(n)$ - It forces the loop to look at everything up to n.
---	---

10. Two-Level Search

<pre>for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { if (arr[i] == target) { break; } } }</pre>	Best : $O(n)$ - If index 0 is a match, it stops instantly. Worst: $O(n^2)$ -It forces the full nested loops to search the entire n square.
---	--

11. Linear Loop With Logarithmic Inner Work

<pre>for (int i = 0; i < n; i++) { int x = n; while (x > 1) { x = x / 2; } }</pre>	Best: $O(n \log n)$ - The outer for loop repeats n times. Inside. Worst: $O(n \log n)$ - Inside, the while loop resets x = n.
---	---

12. Conditional Nested Loop

<pre> if (arr[0] == target) { cout << "Found"; } else { for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { cout << arr[i] + arr[j]; } } } </pre>	Best: $O(1)$ - The index 0 matches, you take the fast if path and completely skip the else blocks. Worst: $O(n^2)$ - Forced into the else block.
---	--

13. Finding the Maximum

<pre> int max = arr[0]; for (int i = 1; i < n; i++) { if (arr[i] > max) { max = arr[i]; } } </pre>	Best: $O(n)$ - The max variable does not stop the loop. Worst: $O(n)$ - The for loop always has to check all the way from index 1 to $n-1$.
---	--

14. Three Nested Loops

<pre> for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { for (int k = 0; k < n; k++) { cout << i + j + k; } } } </pre>	Best: $O(n^3)$ - Three nested loops, where every single loop runs up to n. Worst: $O(n^3)$ - Three nested loops, where every single loop runs up to n.
--	--

15. Comprehensive Analysis

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }

    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Best: $O(1)$ - The Index 0 matches, return instantly.

Worst: $O(n \log n)$ - The outer loop runs n times, and every single time it is forced to do its division.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.